Taller trabajando de manera grupal

Juliana Valentina Dulce Rios

Maria Valentina Arias Cubillos

3406012

Isaura Maria Suarez Novoa

SENA

Introducción

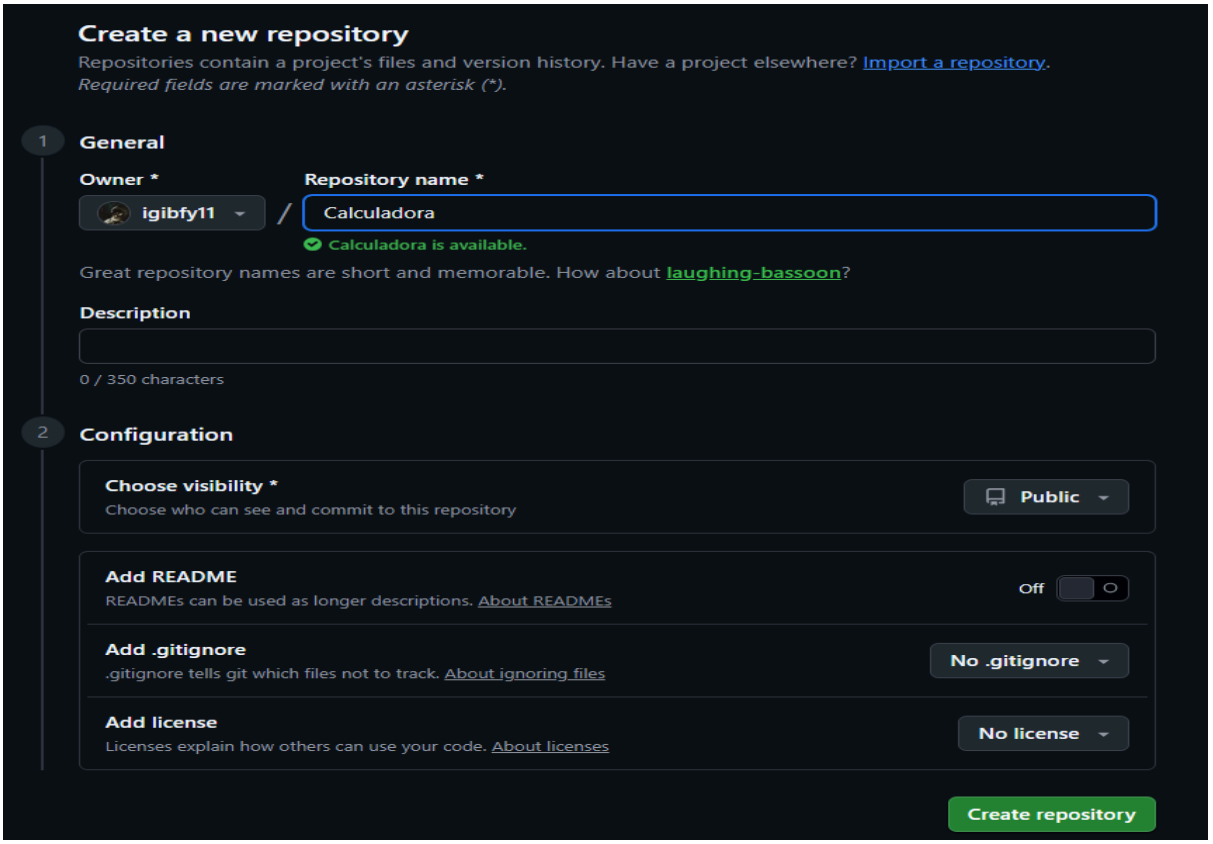
En este documento se registrará el proceso de configuración e implementación de Git Flow en un repositorio en GitHub. Se describirán los procedimientos para iniciar el repositorio, establecer ramas principales y crear una colaboración entre los miembros del equipo. Además de esto se incluirán los comandos que se utilicen, su propósito, los resultados que se obtuvieron y las evidencias visuales que permitirán verificar el desarrollo de esta actividad.

Desarrollo

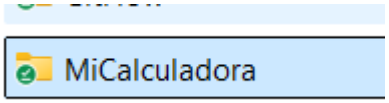
A continuación el paso a paso del desarrollo de la actividad. Ya que el equipo está conformado por dos personas, una de ella será la líder (Juliana) y la otra integrante (María) clonará el repositorio y ejecutará su trabajo. Así que por el momento iniciaremos con el trabajo del líder:

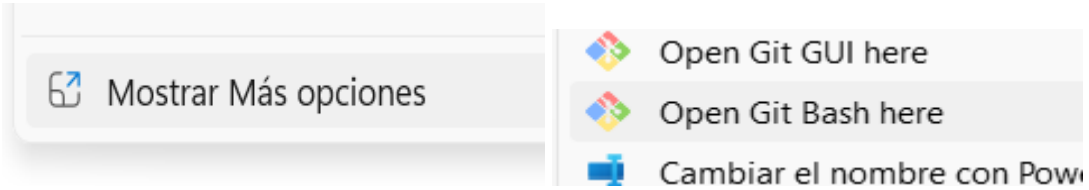
El líder del proyecto:

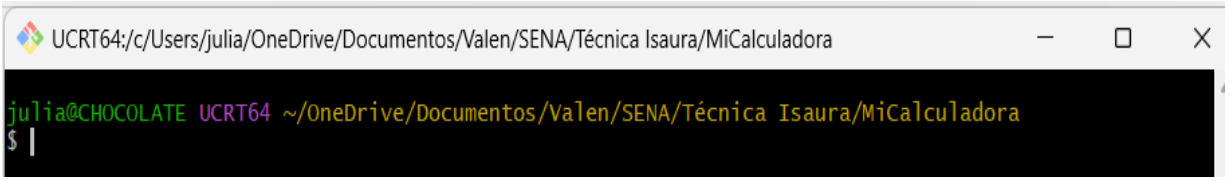
1. Crear el repositorio en GitHub.
   Inicialmente se creó un repositorio en GitHub llamado Calculadora, dejándolo como un repositorio público y con la casilla de Add README sin marcar.



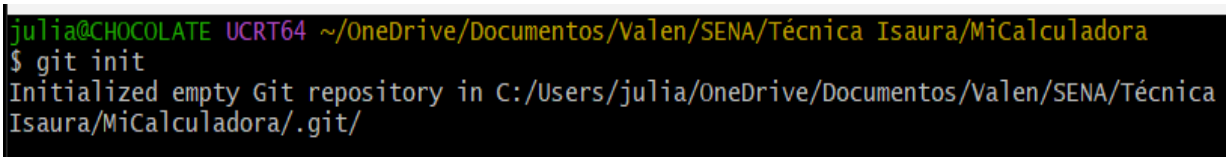
2. Crear la carpeta local y abrir el Git Bash
   Ahora se creó una carpeta en una parte específica del equipo, la cual se nombró MiCalculadora, luego de esto se debe abrir Git Bash haciendo clic derecho, luego clic en Mostrar Más opciones y por último dando clic en Open Git Bash here..



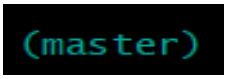


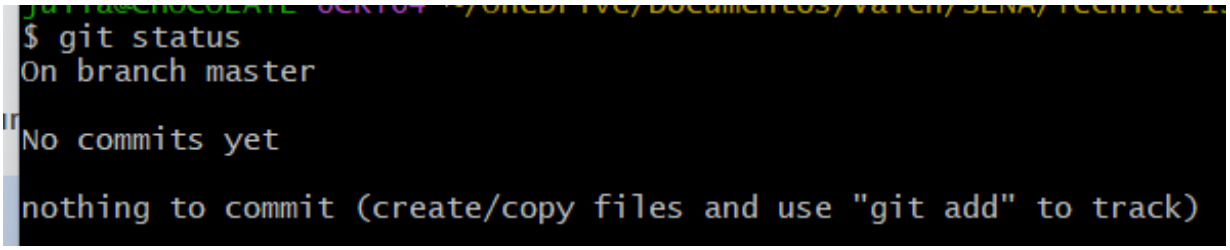


3. Inicializar Git
   En la terminal de Git Bash ejecutamos el siguiente comando para iniciar un repositorio vacío de manera local. Esto creará un archivo oculto con el nombre de .git el cuál almacena la información necesaria para controlar las versiones de los archivos que se trabajen, registrar cambios y gestionar las ramas del proyecto.



   Luego se puede ejecutar *git status* para visualizar el estado de nuestro repositorio local y, en este caso, podremos ver que se creó la rama principal (master) y que en el momento no se ha realizado ningún commit.



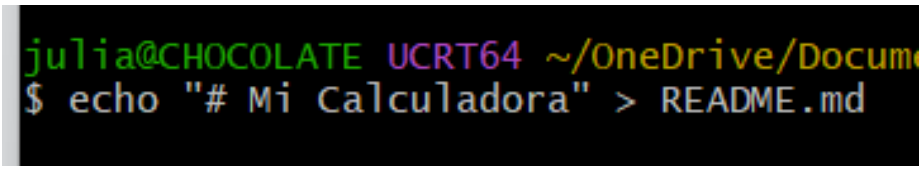


4. Crear un archivo inicial obligatorio y la plantilla para codificar
   Ahora creamos un archivo README.md, este archivo contendrá información básica sobre el proyecto y permite registrar el primer commit, debido a que es un requisito necesario para inicializar Git Flow en el repositorio. Así que ejecutaremos el comando echo "# Mi Calculadora" > README.md

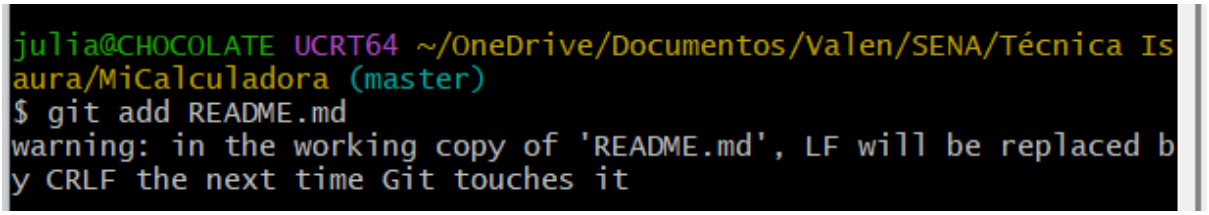
   Lo que está haciendo este comando es:

   - **echo** escribe un texto.
   - "**# Mi Calculadora**" es el contenido que se escribirá en el archivo.
   - **>** crea el archivo README.md o reemplaza su contenido si ya existe.

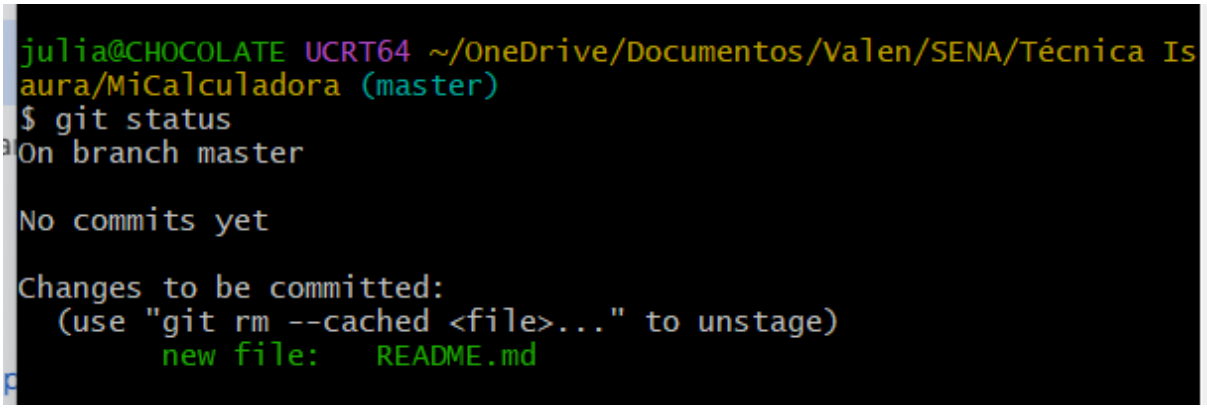
   El símbolo **#** es utilizado en Markdown para representar un título de nivel principal. Markdown es un formato de texto el cual permite organizar información por medio de títulos, listas, enlaces y otros elementos.



   Ahora preparamos el archivo para realizar el commit:



   Ahora por medio del git status podremos ver que el archivo está listo para subirlo.



Para concluir con este punto, realizaremos nuestro primer commit con el mensaje de “plantilla inicial”. Este comando registrará los cambios preparados en el historial del repositorio.

```
julia@CHOCOLATE UCRT64 ~/OneDrive/Documentos/Valen/SENA/Técnica Is
aura/MiCalculadora (master)
$ git commit -m "plantilla inicial"
[master (root-commit) 184f337] plantilla inicial
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

5. Subir al repositorio

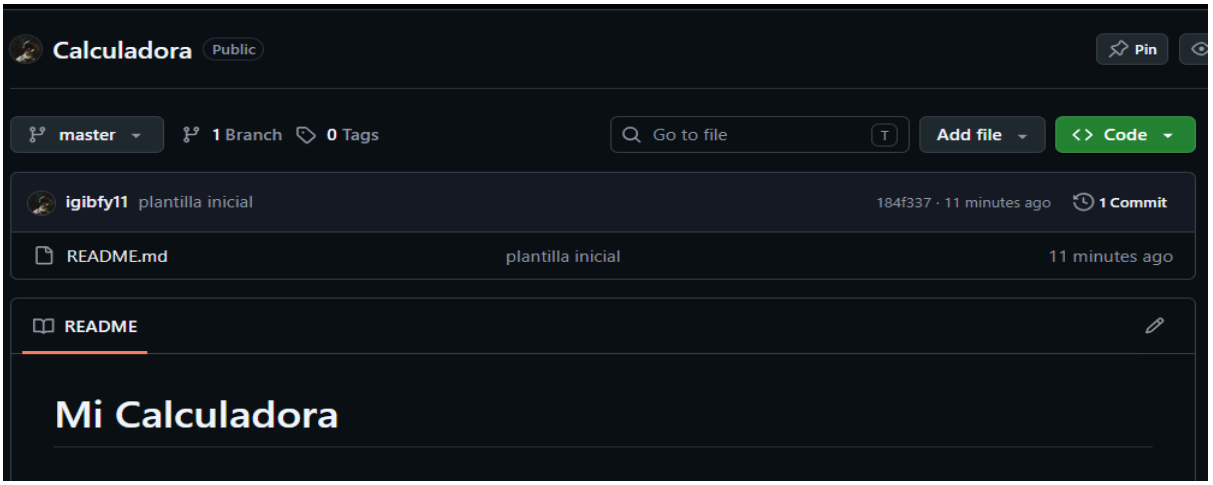
   Ahora por medio del comando git remote add origin …url vamos a conectar nuestro repositorio local con el que previamente creamos en GiHub, haciendo uso de la url de nuestro repositorio, la cual podremos encontrar directamente en GitHub, bien sea de los códigos que aparecen inicialmente o dando clic en el botón verde de Code y copiando el enlace que aparece en este. **Este comando solo se debe ejecutar una vez.**

```
julia@CHOCOLATE UCRT64 ~/OneDrive/Documentos/Valen/SENA/Técnica Isa
ura/MiCalculadora (master)
$ git remote add origin https://github.com/igibfy11/Calculadora.git
```

Continuamos con el comando git push -u origin ramaprincipal el cual sube a GitHub los cambios realizados en el repositorio local. En otras palabras, este comando publica la rama master (o la rama principal) y su historial de commits en GitHub.

```
julia@CHOCOLATE UCRT64 ~/OneDrive/Documentos/Valen/SENA/Técnica Isa
ura/MiCalculadora (master)
$ git push -u origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 240 bytes | 240.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/igibfy11/Calculadora.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

Luego podemos refrescar la página de GitHub y se podrá visualizar que ya se realizó el commit y se subió correctamente.



6. Inicializar Git Flow

   Ahora hay que inicializar Git Flow, pero antes de hacerlo podemos realizar una pequeña verificación de que Git Flow esté descargado, y esto lo haremos por medio del comando git flow version. Cuando aparezca un número de versión, esto significa que está descargado.

```
julia@CHOCOLATE UCRT64 ~/OneDrive/Documentos/Valen/SENA/Técnica Isa
ura/MiCalculadora (master)
$ git flow version
1.12.4-dev0 (AVH Edition)
```

Seguido a esto, ejecutamos el comando git flow init y a cada una de las preguntas que aparezcan les daremos Enter, ya que de esta manera estaremos aceptando todos los valores predeterminados.

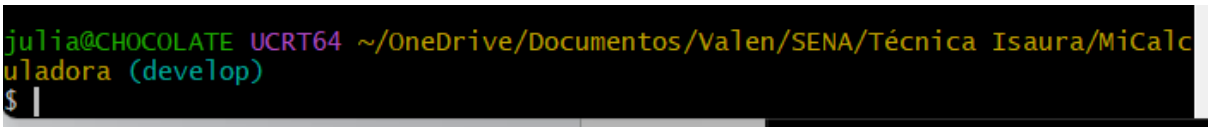
```
julia@CHOCOLATE UCRT64 ~/OneDrive/Documentos/Valen/SENA/Técnica Isaura/MiCalc
uladora (master)
$ git flow init

Which branch should be used for bringing forth production releases?
   - master
Branch name for production releases: [master]
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Bugfix branches? [bugfix/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? []
Hooks and filters directory? [C:/Users/julia/OneDrive/Documentos/Valen/SENA/T
écnica Isaura/MiCalculadora/.git/hooks]
```

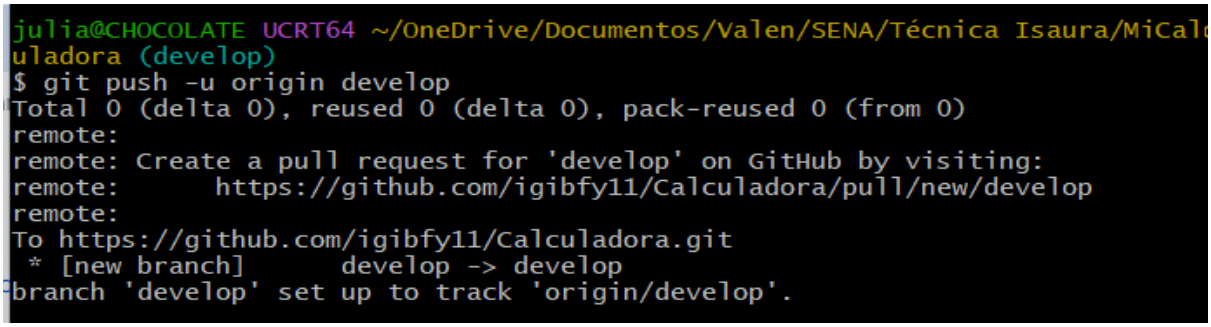
Al inicializar Git Flow, él está identificando a la rama master como la rama de producción y crea la rama develop, a partir de la cual se organizará el desarrollo de nuevas funcionalidades. La rama develop es importante debido a que los integrantes del equipo no deberían desarrollar nada directamente sobre master. En su lugar, crearán ramas feature a partir de develop.

Si observamos bien, luego de inicializar Git Flow, automáticamente nos pasa a la rama develop.

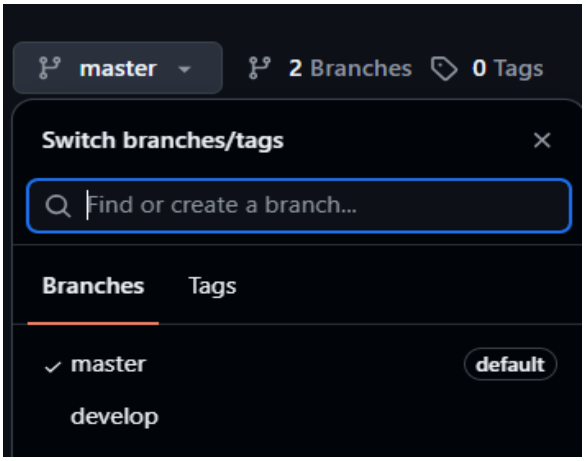


7. Publicar la rama develop en GitHub

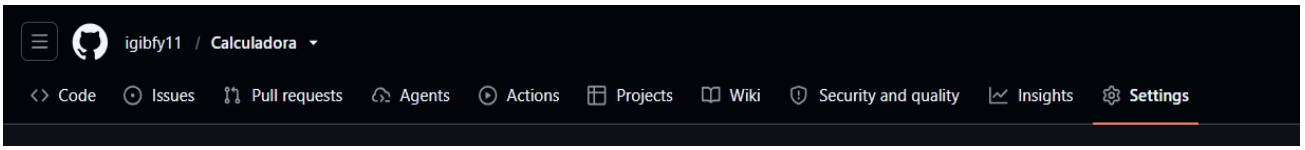
   Aunque Git Flow ya creó la rama develop en tu repositorio local, todavía debemos publicarla en GitHub para que los colaboradores puedan acceder a ella. Esto lo hacemos por medio del comando git push -u origin develop



Este comando publicará la rama develop en el repositorio remoto y establecerá su seguimiento con origin/develop. Después entraremos a Github, refrescamos la página y podremos ver que ya están disponibles las dos ramas, master y develop.

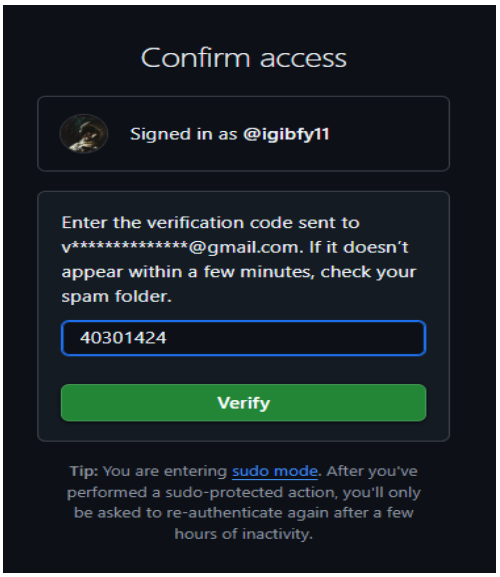


8. Unir al repositorio a los colaboradores
   Ahora el líder del proyecto le enviará la invitación de colaborador a la otra integrante del equipo. Para hacer esto hay que ir a la configuración o seetings del repositorio situado en la parte superior derecha de la vista del repositorio.

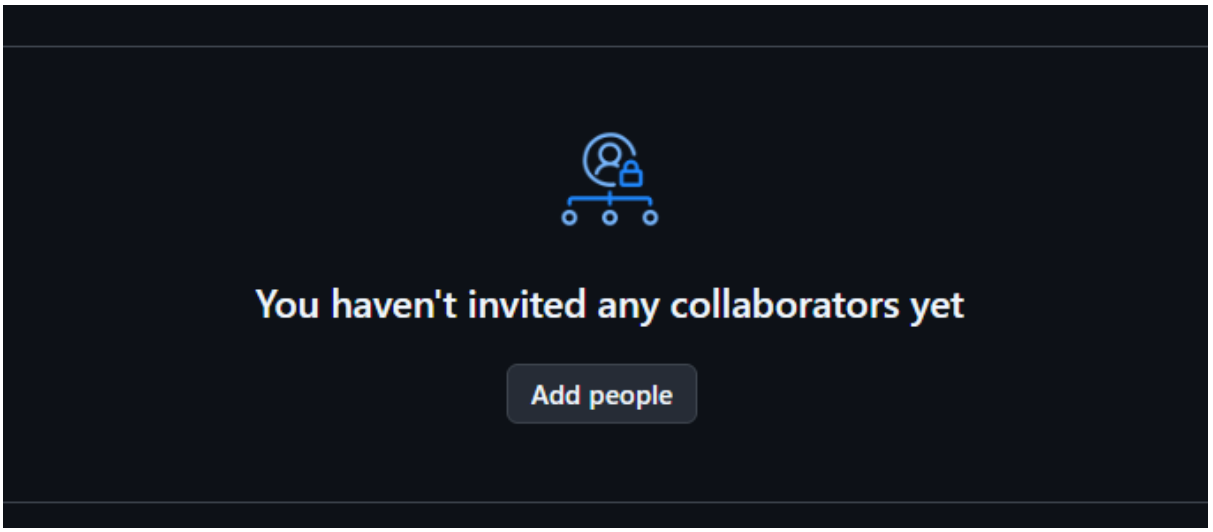


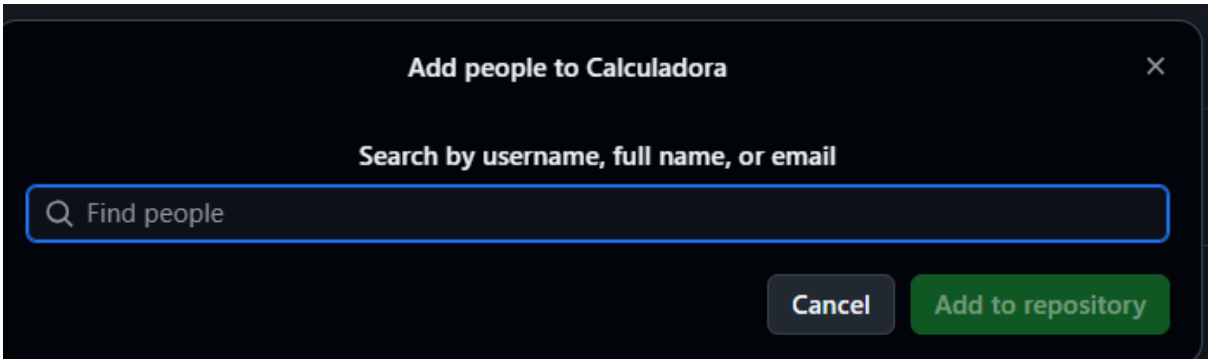
   Luego en el panel lateral seleccionamos la opción de colaboradores/collaborators el cuál pedirá que verifiques tu identidad.





   Luego hacemos clic en el botón de add people y escribiremos el correo de nuestro compañero, luego lo añadimos al repositorio y listo.

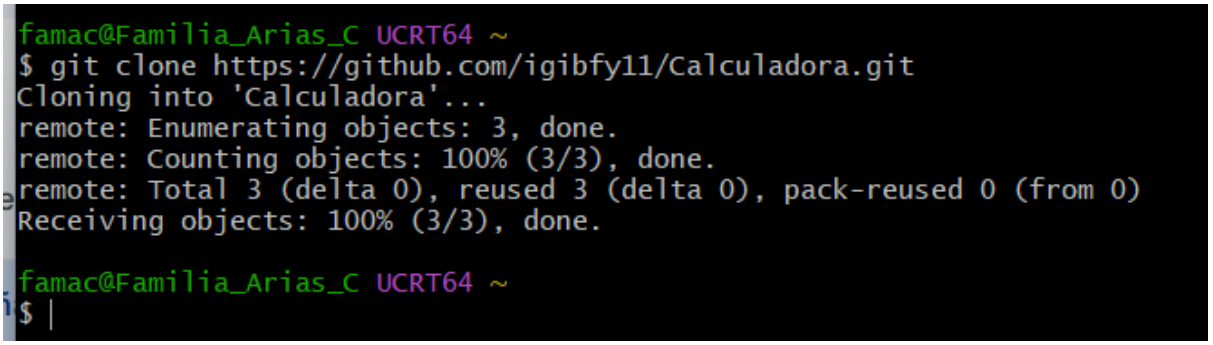




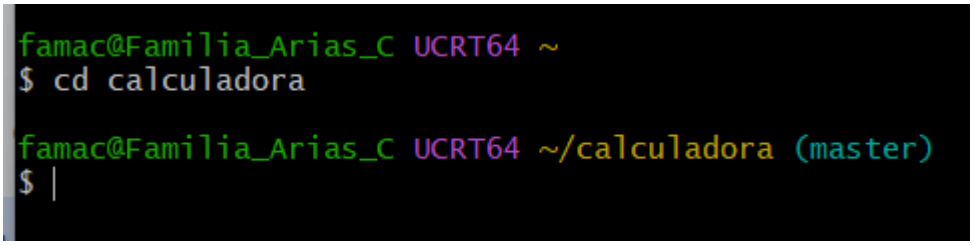
El otro integrante del proyecto:

1. clonar el repositorio:

   git clone URL_DE_TU_REPOSITORIO_AQUÍ



2. abre el bash y asegurate que estas dentro de la carpeta de proyecto



3. inicializa el git flow para que la consola reconozca los comandos, ya sabes que debes presionar enter a todas las preguntas:

   git flow init

```
famac@Familia_Arias_C UCRT64 ~/calculadora (master)
$ git flow init

Which branch should be used for bringing forth production releases?
   - master
Branch name for production releases: [master]



Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?

Feature branches? [feature/]
Bugfix branches? [bugfix/]
Release branches? [release/]

Hotfix branches? [hotfix/]

Support branches? [support/]


Version tag prefix? []

Hooks and filters directory? [C:/Users/famac/calculadora/.git/hooks]
```

4. Debes asegurarte estar en la rama develop

   git checkout develop



en este caso nos aseguramos y ya estaba en la rama entonces no hicimos ningún cambio.

5. Descargar lo último que hayan hecho sus compañeros

   git pull origin develop

```
famac@Familia_Arias_C UCRT64 ~/calculadora (develop)
$ git pull origin develop
From https://github.com/igibfy11/Calculadora
 * branch            develop    -> FETCH_HEAD
Already up to date.
```

6. Crear su rama de funcionalidad:

   git flow feature start login

```
famac@Familia_Arias_C UCRT64 ~/calculadora (develop)
$ git flow feature start login
Switched to a new branch 'feature/login'

Summary of actions:
- A new branch 'feature/login' was created, based on 'develop'
- You are now on branch 'feature/login'

Now, start committing on your feature. When done, use:

     git flow feature finish login
```

7. Codificar, guardar y hacer el commits; y en lugar de finalizar la tarea o funcionalidad localmente, subirla a revisión:

   git flow feature publish login

primero cree un html de restar para esa función de la calculadora dentro del archivo

```
</head>
<body>
    <div style="text-align: center; margin-top: 50px;">
        <h1>Módulo de Operación</h1>
        <input type="number" id="num1" placeholder="Número 1">
        <input type="number" id="num2" placeholder="Número 2">
        <br><br>

        <button onclick="operar()">Calcular</button>
        <h2 id="resultado">Resultado: </h2>
    </div>

    <script>
        function operar() {
            const n1 = parseFloat(document.getElementById('num1').value) || 0;
            const n2 = parseFloat(document.getElementById('num2').value) || 0;

            // Ejemplo con resta (cámbialo por tu operación asignada)
            const resultado = n1 - n2;

            document.getElementById('resultado').innerText = "Resultado: " + resultado;
        }
    </script>
```

después se guarda, y ahora hice el commit que serían los siguientes:

```
famac@Familia_Arias_C UCRT64 ~/calculadora (feature/login)
$ git add .

famac@Familia_Arias_C UCRT64 ~/calculadora (feature/login)
$ git commit -m "agregar la funcionalidad a la calculadora"
[feature/login 3989f8d] agregar la funcionalidad a la calculadora
 1 file changed, 31 insertions(+)
 create mode 100644 restar.html

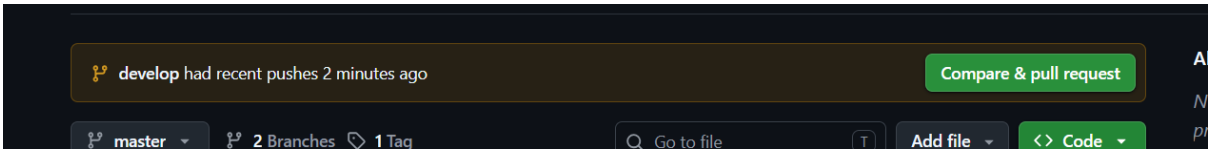
famac@Familia_Arias_C UCRT64 ~/calculadora (feature/login)
$
```

Una vez hecho el commit local, publica tu rama de funcionalidad para que el equipo pueda verla en GitHub

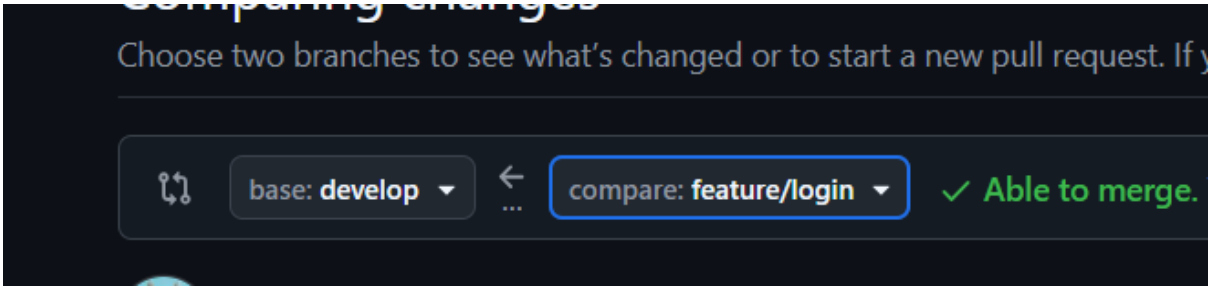
```
famac@Familia_Arias_C UCRT64 ~/calculadora (feature/login)
$ git flow feature publish login
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 849 bytes | 849.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/login' on GitHub by visiting:
remote:      https://github.com/igibfy11/Calculadora/pull/new/feature/login
remote:
To https://github.com/igibfy11/Calculadora.git
 * [new branch]      feature/login -> feature/login
branch 'feature/login' set up to track 'origin/feature/login'.
Already on 'feature/login'
Your branch is up to date with 'origin/feature/login'.

Summary of actions:
- The remote branch 'feature/login' was created or updated
- The local branch 'feature/login' was configured to track the remote branch
- You are now on branch 'feature/login'
```

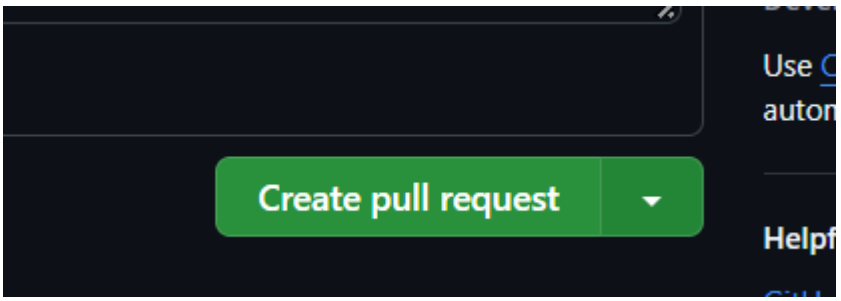
8. Ir a GitHub, abrir un Pull Request hacia la rama develop, esperar la aprobación del equipo y fusionar.



hay que tener muy en cuenta lo siguiente



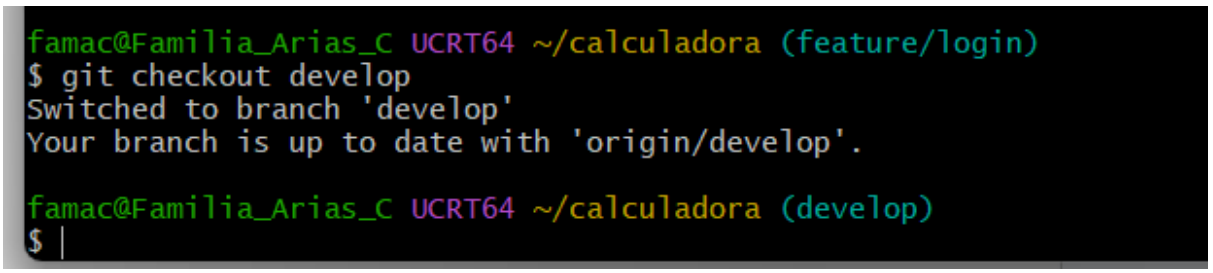
rama receptora (**base**) a develop y como rama de origen (**compare**) a feature/login. Este mecanismo permite la revisión de código por pares (*peer review*), garantizando la validación de la sintaxis y la ausencia de conflictos antes de su fusión definitiva (*merge*) en el repositorio del equipo.



Esto lo hacemos en equipo para que mis compañeros puedan revisar que todo funcione bien y aprueben los cambios antes de juntarlos con el proyecto principal

9. Despues que tu equipo aprueba tu pull request, debes finalizar tu funcionalidad, para ello debes estar en la rama develop, en caso de no estes, debes colocarte en esa rama:

   git checkout develop



10. trae los cambios

    git pull origin develop

```
famac@Familia_Arias_C UCRT64 ~/calculadora (develop)
$ git pull origin develop
From https://github.com/igibfy11/Calculadora
 * branch            develop    -> FETCH_HEAD
Already up to date.

famac@Familia_Arias_C UCRT64 ~/calculadora (develop)
$
```

11. cierra tu tarea o funcionalidad

    git flow feature finish <nombre-de-tu-tarea o funcionalidad>

```
famac@Familia_Arias_C UCRT64 ~/calculadora (develop)
$ git flow feature finish login
Already on 'develop'
Your branch is up to date with 'origin/develop'.
Updating 184f337..3989f8d
Fast-forward
 restar.html | 31 ++++++++++++++++++++++++++++++
 1 file changed, 31 insertions(+)
 create mode 100644 restar.html
To https://github.com/igibfy11/Calculadora.git
 - [deleted]         feature/login
Deleted branch feature/login (was 3989f8d).

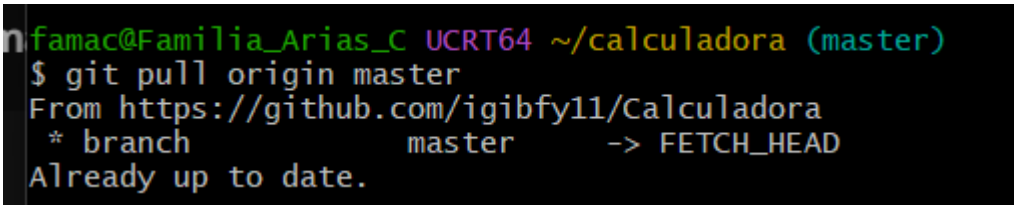
Summary of actions:
- The feature branch 'feature/login' was merged into 'develop'
- Feature branch 'feature/login' has been locally deleted; it has been remotely deleted from 'origin
'
```

12. sube a la rama master y develop, Asegurar que develop esté al día en la nube, Esto confirma que cualquier cambio pendiente en tu develop local suba a GitHub

    Sincronizar la rama master

```
famac@Familia_Arias_C UCRT64 ~/calculadora (develop)
$ git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.

famac@Familia_Arias_C UCRT64 ~/calculadora (master)
$
```



Finalmente, se ejecutó git flow feature finish para eliminar de manera limpia la rama temporal local, manteniendo el historial del repositorio ordenado y sincronizado con las ramas principales develop y master

Link repositorio: https://github.com/igibfy11/Calculadora.git